Summer Internship Report on

TOXIGUARD: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted By

Anumathi Singh

Roll no: 160125862006

Under the Esteemed Internal Guidance of

Dr. B. Indira

Associate Professor

Department of MCA, CBIT (A)



Department of MCA

Chaitanya Bharathi Institute of Technology (Autonomous)

Gandipet, Hyderabad — Telangana

Academic Year: 2026–27

CERTIFICATE

This is to certify that the Internship Report titled "ToxiGuard: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection" is a bona fide record of work carried out by Anumathi Singh (Hall Ticket No. 160125862006), in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA).

The report presents the ToxiGuard project as part of the summer internship requirement following the Artificial Intelligence program at EduExpose.

[Internship period: 04-07-2026 – 15-08-2026]

Internship Internal Guide	Head of the Department
Dr. B. Indira	**Dr. B. Indira**
Associate Professor	**Associate Professor**
Dept. of MCA, CBIT (A), Hyderabad	**Dept. of MCA, CBIT (A), Hyderabad**

ORGANIZATION CERTIFICATE

The completion certificate issued by EduExpose for the Artificial Intelligence internship program is reproduced below.



DECLARATION

I, Anumathi Singh, bearing Hall Ticket No. 160125862006, hereby declare that the internship project report entitled “ToxiGuard: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection”, submitted as part of the requirements of the Master of Computer Applications (MCA) program, is an original work carried out by me under the guidance of my internal guide, Dr. B. Indira.

I further declare that this report presents the design, implementation, testing, and documentation of the ToxiGuard toxic-comment detection system. The report has not been previously submitted, either in whole or in part, to any institution or university for the award of any degree, diploma, certification, or academic credit.

All tools, libraries, datasets, and technical references used in the preparation of this report have been acknowledged to the best of my knowledge and belief.

Anumathi Singh

Roll No: 160125862006

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me in completing my internship project titled “ToxiGuard: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection”.

I am deeply grateful to Dr. GNR Prasad, my internship guide at Chaitanya Bharathi Institute of Technology (CBIT), for continuous encouragement and valuable suggestions.

I am deeply grateful to Dr. B. Indira, my college mentor at Chaitanya Bharathi Institute of Technology (CBIT), for continuous encouragement and valuable suggestions.

I am grateful to Dr. B. Indira, Head of the MCA Department at Chaitanya Bharathi Institute of Technology (CBIT), for her guidance, support, and encouragement.

I also thank Dr. C. V. Narasimhulu, Principal, Chaitanya Bharathi Institute of Technology, for his support in this endeavor.

I extend my heartfelt thanks to EduExpose for their valuable guidance and support throughout the project.

Finally, I thank the faculty members of the Department of MCA, and my family and friends, for their constant encouragement and support throughout the project.

Anumathi Singh

ABSTRACT

ToxiGuard is an online comment analysis application that detects potentially toxic language and presents predictions for human review. It uses natural language processing and machine learning to classify comments as toxic or non-toxic. The application is developed in Python and Streamlit, using the Jigsaw toxic-comment dataset. Text preprocessing and TF-IDF feature extraction convert comments into a shared numerical representation for model training and prediction.

Six algorithms, namely Multinomial Naive Bayes, Complement Naive Bayes, Logistic Regression, calibrated Linear SVM, SGD Classifier, and Random Forest, were compared using accuracy, precision, recall, F1-score, and ROC-AUC. A stratified subset of 30,000 comments was divided into 24,000 training comments and a 6,000-comment model-selection holdout. Logistic Regression achieved the highest toxic-class F1 score of 69.75%, with 93.23% accuracy, 61.02% precision, 81.39% recall, and 0.9576 ROC-AUC.

The application provides six pages: Overview, Analyze comment, Batch analysis, Model performance, Dataset insights, and About the project. Users can submit individual comments, upload a CSV or enter multiple comments, download predictions, and inspect the saved model results. Each prediction includes a class, toxic probability, displayed confidence, and review-priority band. The same text-cleaning function and fitted vectorizer are used during training and inference.

The report describes the system analysis, design, implementation, testing, results, and limitations of ToxiGuard. The project connects model comparison with a working moderation-support interface. Its predictions concern individual comments; conversation context, intent, and repeated bullying require human assessment. The evaluation does not establish performance on an independent final test set or fairness across social groups.

Keywords: toxic comment detection, natural language processing, TF-IDF, Logistic Regression, machine learning, Streamlit.

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Introduction to ToxiGuard 1
1.2 Project Objectives 1
1.3 Scope of the Project 2
2. ORGANIZATION PROFILE 3
2.1 Internship Program 3
2.2 Academic Setting 3
2.3 Relevance of the Program 3
3. LITERATURE REVIEW 4
3.1 Automated Analysis of Harmful Comments 4
3.2 Distinguishing Toxicity and Offensive Language 4
3.3 Bias in Text Classification 4
3.4 Relevance to the Proposed System 4
4. SYSTEM ANALYSIS 5
4.1 Problem Statement 5
4.2 Existing System and Limitations 5
4.3 Proposed System 5
4.4 Functional Requirements 5
4.5 Non-Functional Requirements 6
4.6 Hardware and Software Requirements 6
4.7 System Boundaries 6
5. TECHNOLOGIES AND TOOLS USED 7
6. SYSTEM DESIGN 8
6.1 Data Flow Diagram 8
6.2 Entity Relationship Diagram 10
7. IMPLEMENTATION 11
7.1 Modules 11
7.2 Data Preparation 11
7.3 Feature Extraction and Training 11
7.4 Selection and Serialization 11
7.5 Streamlit Application 12

7.6 Single-Comment Prediction 13
7.7 Batch Analysis and Export 13
7.8 Review Priority and Confidence 13
8. TESTING AND VALIDATION 14
8.1 Test Strategy 14
8.2 Functional Checks 14
8.3 Interpretation 14
9. RESULTS AND DISCUSSION 15
9.1 Dataset and Experiment 15
9.2 Model Comparison 16
9.3 Application Output 19
9.4 Limitations 20
10. CHALLENGES AND SOLUTIONS 21
10.1 Class Imbalance 21
10.2 Training and Inference Consistency 21
10.3 Interpretation and Context 21
11. LEARNING OUTCOMES 22
11.1 Technical Skills Acquired 22
11.2 Practical Experience Gained 22
11.3 Technical Communication 22
12. CONCLUSION AND FUTURE SCOPE 23
12.1 Conclusion 23
12.2 Limitations 23
12.3 Future Enhancements 23
REFERENCES 24
Appendix A — Source Code 25
Appendix B — Paper Presentation Certificate 29
Appendix C — Research Paper 30
Appendix D — Project Outputs 40

LIST OF FIGURES

Figure 6.1: DFD Level 0 — ToxiGuard Context....8

Figure 6.2: DFD Level 1 — Training and Prediction....9

Figure 6.3: Logical Entity Relationship Model....10

Figure 7.1: Streamlit Application Overview....12

Figure 9.1: Exploratory Data Analysis....15

Figure 9.2: Comparison of Classifier Performance....17

Figure 9.3: Logistic Regression Confusion Matrix....18

Figure 9.4: Single-Comment Prediction Output....19

LIST OF TABLES

Table 4.1: Functional Requirements 5

Table 5.1: Technologies and Tools Used 7

Table 7.1: Main Project Modules 11

Table 8.1: Functional Test Cases 14

Table 9.1: Model Comparison 16

Table 9.2: Sample Prediction Results 20

Table D.1: Project Output Files 40

1. INTRODUCTION

1.1 Introduction to ToxiGuard

ToxiGuard is a natural language processing and machine learning system that identifies potentially toxic language in online comments. It accepts a comment as text and predicts whether it belongs to the toxic or non-toxic class. Online discussions may contain insults, hostile remarks, and other harmful expressions that are difficult to review individually when comment volumes are large. ToxiGuard provides a consistent first-stage screening tool that helps a reviewer identify comments requiring closer attention.

The project implements a complete text-classification workflow in Python. Labelled comments from the Jigsaw dataset are cleaned and converted into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). Six machine learning classifiers are compared using the same training and test split. The selected Logistic Regression model is integrated into a Streamlit application that supports individual comment analysis, CSV batch predictions, dataset exploration, and model-performance review.

ToxiGuard addresses harmful language at the comment level. Its toxic-class probability can help prioritise moderation, but a single comment does not establish a pattern of cyberbullying or reveal the full context of a conversation. The application therefore presents predictions as support for human review. The work documented here includes dataset preparation, model comparison, system design, implementation, evaluation, and the application interface.

1.2 Project Objectives

- Prepare a labelled English toxic-comment dataset and examine class balance.
- Implement consistent text preprocessing and TF-IDF feature extraction.
- Compare six classifiers using accuracy, precision, recall, F1-score, and ROC-AUC.
- Integrate the selected model into Streamlit for single-comment and batch analysis.
- Present predictions, evaluation results, and model limitations clearly.

1.3 Scope of the Project

The current scope is English-language binary classification. It does not classify the separate Jigsaw categories in the user interface, use conversation history, or automate content removal.

The deliverables include the cleaned dataset, preprocessing and training code, a training notebook, saved model and vectorizer, evaluation tables and charts, prediction exports, and the Streamlit application. The measured results describe a stratified sample from the source dataset; testing with comments from other platforms is part of future work.

2. ORGANIZATION PROFILE

2.1 Internship Program

EduExpose provided the Artificial Intelligence program completed by Anumathi Singh from 4 July 2026 to 15 August 2026. The program forms the internship context for this academic submission. ToxiGuard applies machine learning concepts to the practical problem of identifying toxic online comments.

2.2 Academic Setting

The work is submitted to the Department of MCA, Chaitanya Bharathi Institute of Technology (Autonomous), Gandipet, Hyderabad. Dr. B. Indira serves as both the Internal Guide and Head of Department for this submission. The project connects natural language processing, experimental model comparison, and a working web interface.

2.3 Relevance of the Program

The Artificial Intelligence program is directly relevant to text classification. ToxiGuard demonstrates data cleaning, feature extraction, supervised learning, metric selection, and deployment. The report emphasizes what is present in the project repository and what the measured outputs actually show.

3. LITERATURE REVIEW

3.1 Automated Analysis of Harmful Comments

Wulczyn, Thain, and Dixon combined crowdsourced labels with machine learning to study personal attacks in English Wikipedia discussions [6]. Their work demonstrates how labelled comments can support automated analysis at a scale that is difficult to achieve through manual review alone. This provides relevant background for ToxiGuard's use of supervised learning on online comments.

3.2 Distinguishing Toxicity and Offensive Language

Davidson and colleagues studied the distinction between hate speech, offensive language, and comments belonging to neither category [7]. Their analysis shows why the presence of a flagged word alone is an inadequate basis for classifying a message. ToxiGuard uses the dataset's binary toxic label, so its output must not be interpreted as a separate determination of hate speech, threat, or repeated cyberbullying.

3.3 Bias in Text Classification

Dixon and colleagues examined unintended bias in toxicity classifiers and showed how imbalanced training examples can affect predictions associated with demographic identity terms [8]. Their work motivates evaluation beyond an overall accuracy score. ToxiGuard reports toxic-class precision, recall, and F1, but its current experiment does not establish fairness across demographic groups; targeted bias evaluation remains future work.

3.4 Relevance to the Proposed System

The reviewed work highlights the value of labelled data, careful interpretation of class definitions, and explicit analysis of model errors. ToxiGuard applies these ideas through a reproducible TF-IDF baseline, comparison of six classifiers, a confusion matrix, and a Streamlit interface for inspecting predictions. Its contribution is an implemented and evaluated academic application. Conversation-level analysis, cross-platform validation, and subgroup evaluation are outside the current experimental scope.

4. SYSTEM ANALYSIS

4.1 Problem Statement

The project addresses the need to screen English online comments for potentially toxic language. The system must accept text, apply the same processing used during training, and return an interpretable binary prediction. Because non-toxic comments form the majority of the dataset, evaluation must also show how well the model detects the smaller toxic class.

4.2 Existing System and Limitations

Manual moderation requires reviewers to inspect individual comments. This provides context, but becomes time consuming as comment volume grows. A simple keyword list is quick but cannot reliably handle variation in wording, negation, or benign uses of flagged terms.

4.3 Proposed System

ToxiGuard applies the same preprocessing used during training to a new comment, transforms it with the saved TF-IDF vectorizer, and produces a toxic-class probability from the selected Logistic Regression classifier. The dashboard displays the prediction, probability, and review priority. It also supports CSV-based batch analysis.

4.4 Functional Requirements

Table 4.1: Functional Requirements

Requirement	Implementation
Input	Accept one comment or a CSV batch in the dashboard
Preparation	Apply the same cleaning function to training and new text
Prediction	Return a binary class and toxic probability
Evaluation	Display model comparison and confusion matrix
Export	Save test predictions and allow batch download

4.5 Non-Functional Requirements

The interface should be legible and usable on a normal laptop. Training results should be reproducible from the documented CSV and fixed random seed. The application should load the same vectorizer and selected model that were saved by training. Predictions are decision support and should be accompanied by a human-review warning.

4.6 Hardware and Software Requirements

A standard Windows laptop with Python 3 and enough free storage for the CSV, model files, and generated charts is sufficient. The project requirements list Streamlit, pandas, NumPy, scikit-learn, joblib, matplotlib, seaborn, Jupyter, and ReportLab. The dashboard is launched with python -m streamlit run app.py from the ToxiGuard directory.

4.7 System Boundaries

The model learns from labelled Wikipedia comments rather than every social platform. Sarcasm, quotations, identity terms, code switching, and changing slang can produce errors. The reported test metrics describe performance on the selected held-out dataset, not guaranteed performance in deployment.

5. TECHNOLOGIES AND TOOLS USED

Table 5.1: Technologies and Tools Used

Component	Role in ToxiGuard
Python	Training script, preprocessing, and dashboard logic
pandas and NumPy	Tabular cleaning, analysis, and prediction export
scikit-learn	TF-IDF, train/test split, classifiers, and metrics
matplotlib and seaborn	EDA, comparison chart, and confusion matrix
joblib	Saving and loading the selected model and vectorizer
Streamlit	Interactive single and batch comment analysis
Jigsaw CSV	Labelled English comment source

The project uses files rather than a relational database. The source and cleaned records are CSV files; trained artifacts are serialized as PKL files. The application reads these outputs from the local project directory.

6. SYSTEM DESIGN

6.1 Data Flow Diagram

The context diagram shows the main inputs and outputs of the system. The level 1 diagram separates the training path from prediction. Both paths use the same cleaning rules; the prediction path loads the saved vectorizer and classifier.

Figure 6.1: DFD Level 0 — ToxiGuard Context

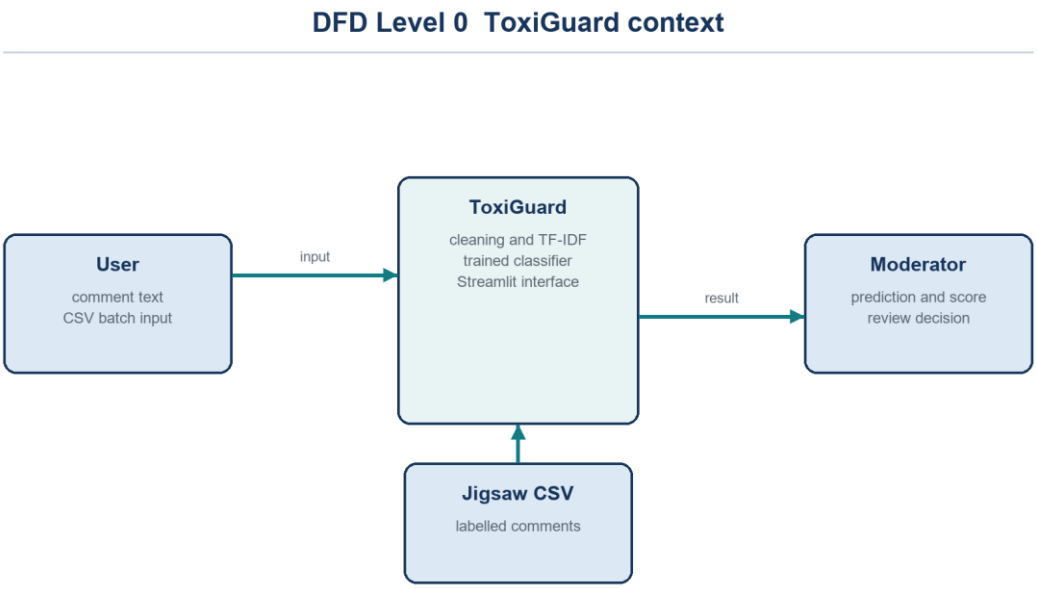


Figure 6.1 represents ToxiGuard as a single process. The user supplies a comment or a batch of comments, while the Jigsaw dataset supplies labelled training examples. The system returns a predicted class and probability to support a moderator's review.

Figure 6.2: DFD Level 1 — Training and Prediction

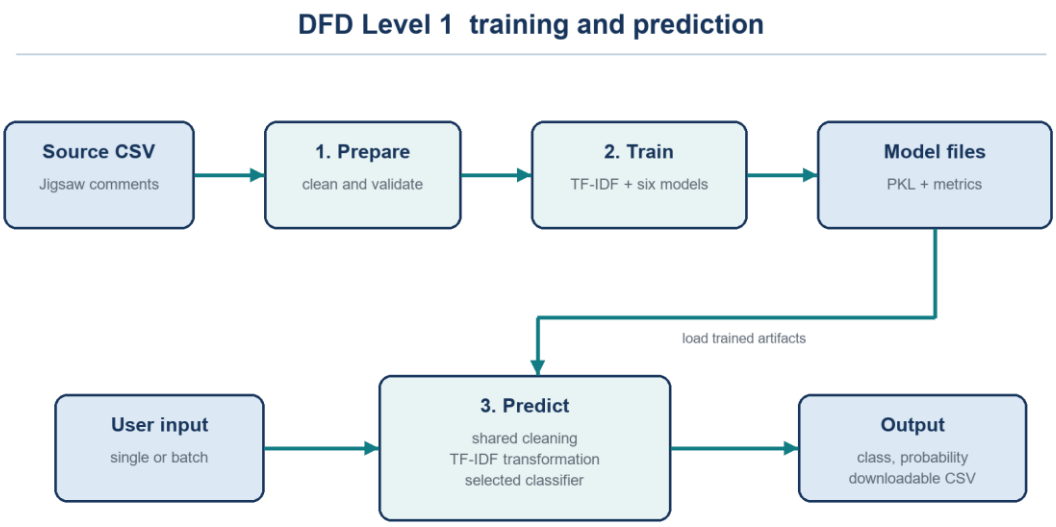


Figure 6.2 separates data preparation, model training, and prediction. The training path cleans the source comments, creates TF-IDF features, compares six classifiers, and saves the selected model. The prediction path applies the same cleaning rules to user input and loads the saved artifacts to generate a class and probability.

6.2 Entity Relationship Diagram

The ER diagram is a logical description of the data and evaluation records. It does not imply a database implementation. The project stores comments and predictions as CSV files and model artifacts as PKL files.

Figure 6.3: Logical Entity Relationship Model

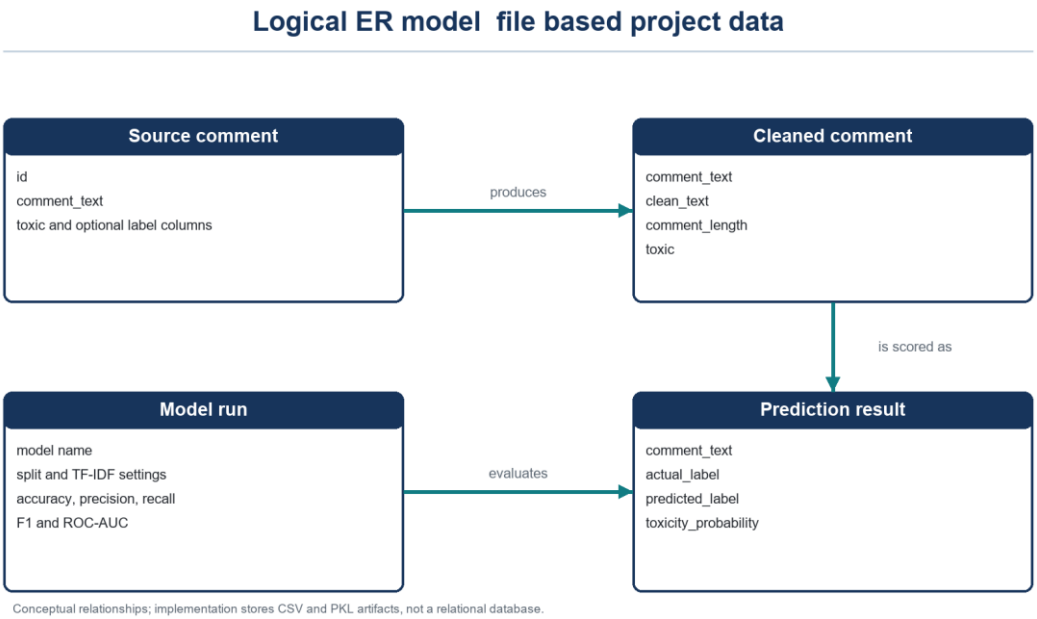


Figure 6.3 relates source comments, cleaned comments, model runs, and prediction results. Cleaning creates the text representation used by the classifier. A model run records evaluation metrics, and prediction records contain the actual label, predicted label, and toxic probability. These are conceptual entities represented by the project's files.

7. IMPLEMENTATION

7.1 Modules

Table 7.1: Main Project Modules

Module	Purpose
download_dataset.py	Obtains the public source CSV
src/text_utils.py	Provides text cleaning and label helpers
train_model.py	Cleans, trains, compares, and exports outputs
app.py	Loads saved artifacts and presents predictions

7.2 Data Preparation

The training script reads comment_text and toxic, removes rows with missing required values and duplicate comment text, applies the shared cleaning function, and removes records whose cleaned text is too short. It records comment length for EDA. The reported cleaned dataset contains 159,556 rows, including 15,293 toxic comments and 144,263 non-toxic comments.

7.3 Feature Extraction and Training

A stratified sample of 30,000 rows is split 80:20 with random state 42. The TF-IDF vectorizer is fitted on training text only and transforms the test text without refitting. It uses unigrams and bigrams, min_df=2, sublinear term frequency, and a 15,000-feature cap. Six classifiers receive the same feature matrix.

7.4 Selection and Serialization

Each model is assessed with accuracy, toxic-class precision, recall, F1, and ROC-AUC. The comparison is sorted by toxic-class F1, then recall, then accuracy. The selected Logistic Regression model and TF-IDF vectorizer are saved separately with joblib. The output folder holds the comparison table, confusion matrix, classification report, prediction export, EDA plots, and metadata.

7.5 Streamlit Application

The dashboard offers Overview, Analyze comment, Batch analysis, Model performance, Dataset insights, and About the project pages. Its Analyze comment page takes a text input and displays a class, toxic probability, model confidence, and review priority. The Batch analysis page accepts comments in a CSV and exports predicted results.

Figure 7.1: Streamlit Application Overview

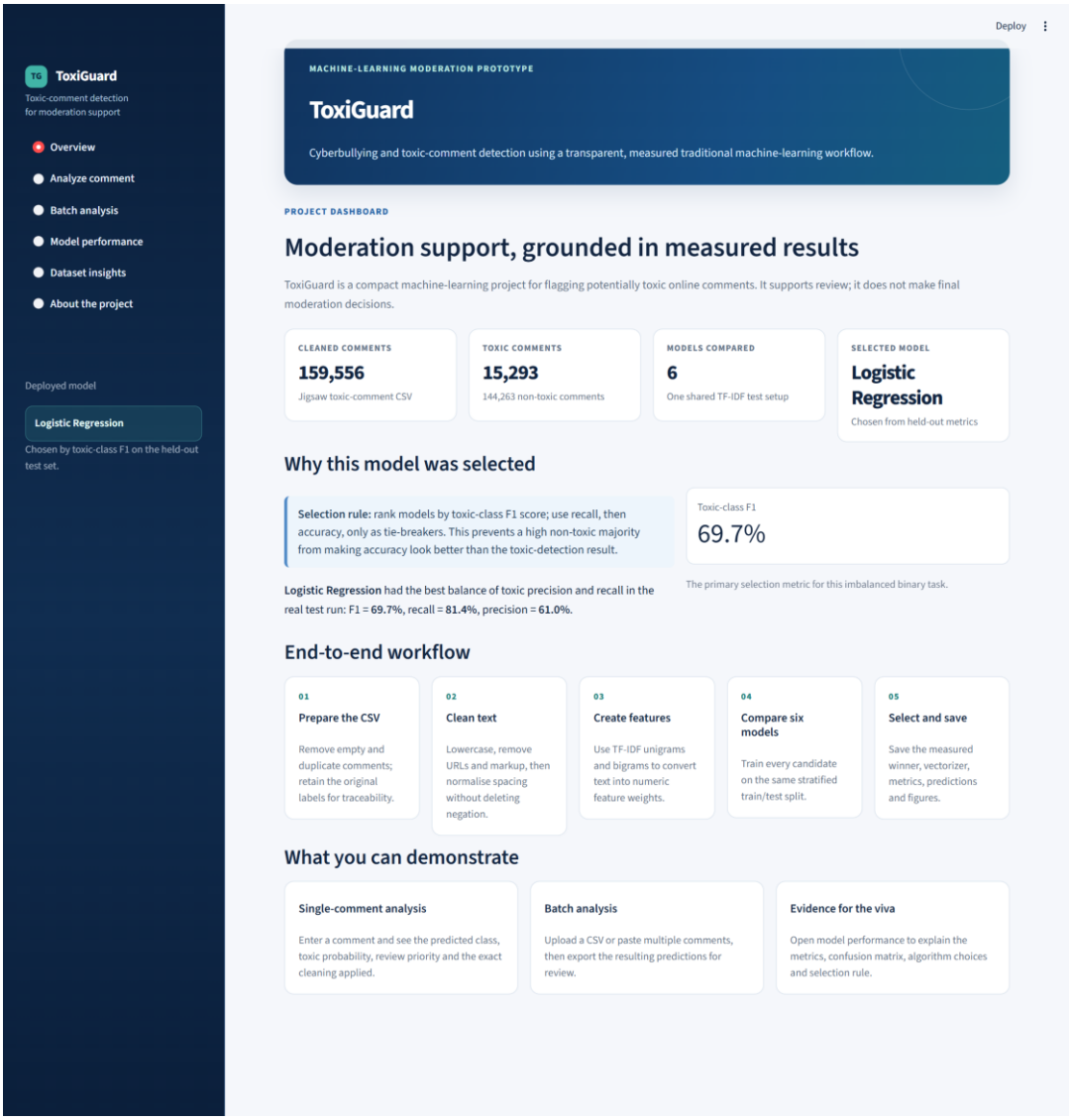


Figure 7.1 shows the dashboard overview, including the cleaned dataset size, toxic-comment count, six-model comparison, and selected Logistic Regression model. The sidebar provides access to comment analysis, batch analysis, model performance, and dataset insights.

7.6 Single-Comment Prediction

A submitted comment is cleaned and transformed with the saved TF-IDF vectorizer before the classifier produces its toxic probability. A probability of at least 0.50 gives the Toxic label. Empty cleaned input produces a warning. For a linear model with available coefficients, the interface can display present features with positive contributions to the fitted score. These contributions describe model behaviour and do not establish the intent of the writer.

7.7 Batch Analysis and Export

The batch interface accepts a CSV file or one comment per line. For a CSV, the user selects the text column, with comment_text preferred when present. Missing values are removed and comments that become empty after cleaning are skipped. Each operation processes at most the first 1,000 submitted comments. The downloadable CSV includes the comment, prediction, toxic probability, confidence, and review priority. Recent checks are held in the session rather than a permanent annotation database.

7.8 Review Priority and Confidence

Review priority is assigned separately from the binary prediction: a probability below 0.45 gives Lower review priority; a probability from 0.45 to below 0.75 gives Review recommended; and a probability of at least 0.75 gives High review priority. Consequently, a comment can receive a non-toxic label and still be recommended for review. These bands are interface rules rather than validated measures of harm severity. Displayed confidence is the larger of the toxic and non-toxic probabilities, and is not a guarantee that the prediction is correct.

8. TESTING AND VALIDATION

8.1 Test Strategy

The training script was run against the local Jigsaw CSV and completed with six measured model results. The dashboard was started locally and its Overview, Analyze comment, and Model performance pages were inspected. Sample comments were evaluated through the saved artifacts using the same cleaning and vectorization path.

8.2 Functional Checks

Table 8.1: Functional Test Cases

ID	Input or action	Observed result
T01	Start Streamlit dashboard	Application loaded with Logistic Regression selected
T02	Insulting sample comment	Toxic, probability 0.9915
T03	Appreciative sample comment	Non-toxic, probability 0.1427
T04	Polite disagreement	Non-toxic, probability 0.0501
T05	Open model performance	Six-model comparison and selected metrics displayed

8.3 Interpretation

These checks confirm that the saved artifacts load and the interface produces a result for representative input. Three sample comments are not a statistical validation set; the 6,000-row held-out evaluation provides the quantitative evidence. A future acceptance test should include human-reviewed comments from the intended deployment setting.

9. RESULTS AND DISCUSSION

9.1 Dataset and Experiment

The cleaned dataset contains 159,556 comments, with the toxic class representing about 9.6% of rows. A stratified 30,000-row subset was divided into 24,000 training comments and 6,000 test comments. This imbalance is why accuracy alone is insufficient for model selection.

Figure 9.1: Exploratory Data Analysis

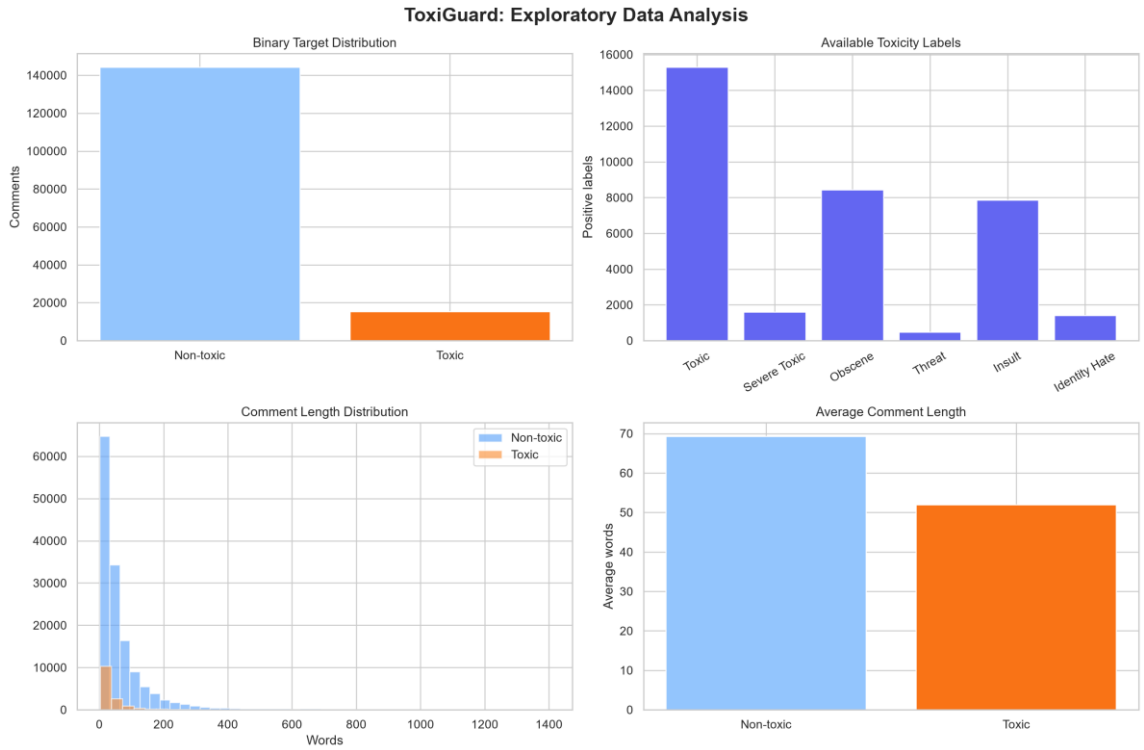


Figure 9.1 shows four views of the cleaned dataset: binary class counts, counts for the available toxicity labels, comment-length distributions, and average comment length. Non-toxic comments greatly outnumber toxic comments. The skewed length distribution and unequal class sizes provide context for preprocessing and minority-class evaluation.

9.2 Model Comparison

Table 9.1: Model Comparison

Model	Acc	Prec	Recall	F1	AUC
Logistic Regression	0.932	0.610	0.814	0.697	0.958
SGD Classifier	0.932	0.613	0.795	0.692	0.953
Linear SVM (Calibrated)	0.950	0.855	0.574	0.687	0.956
Multinomial Naive Bayes	0.945	0.926	0.459	0.614	0.945
Complement Naive Bayes	0.892	0.467	0.864	0.606	0.945
Random Forest	0.917	0.558	0.628	0.591	0.914

Table 9.1 reports held-out accuracy (Acc), toxic precision (Prec), toxic recall, toxic F1, and ROC-AUC (AUC). Logistic Regression achieved the highest toxic-class F1 of 0.6975. It recovered about 81.4% of toxic comments at a toxic precision of about 61.0%. The calibrated Linear SVM had higher overall accuracy and precision, but lower toxic recall. The choice therefore reflects the documented F1-first selection rule.

Figure 9.2: Comparison of Classifier Performance

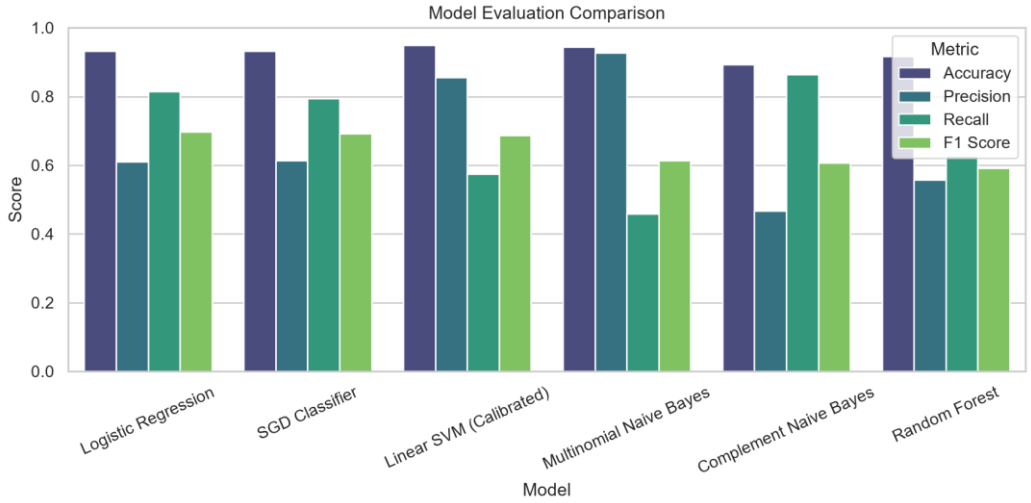


Figure 9.2 compares accuracy, toxic precision, recall, and F1 across six classifiers. Logistic Regression achieves the highest toxic F1. Complement Naive Bayes has higher recall but more false positives, while Multinomial Naive Bayes misses more toxic comments.

Figure 9.3: Logistic Regression Confusion Matrix

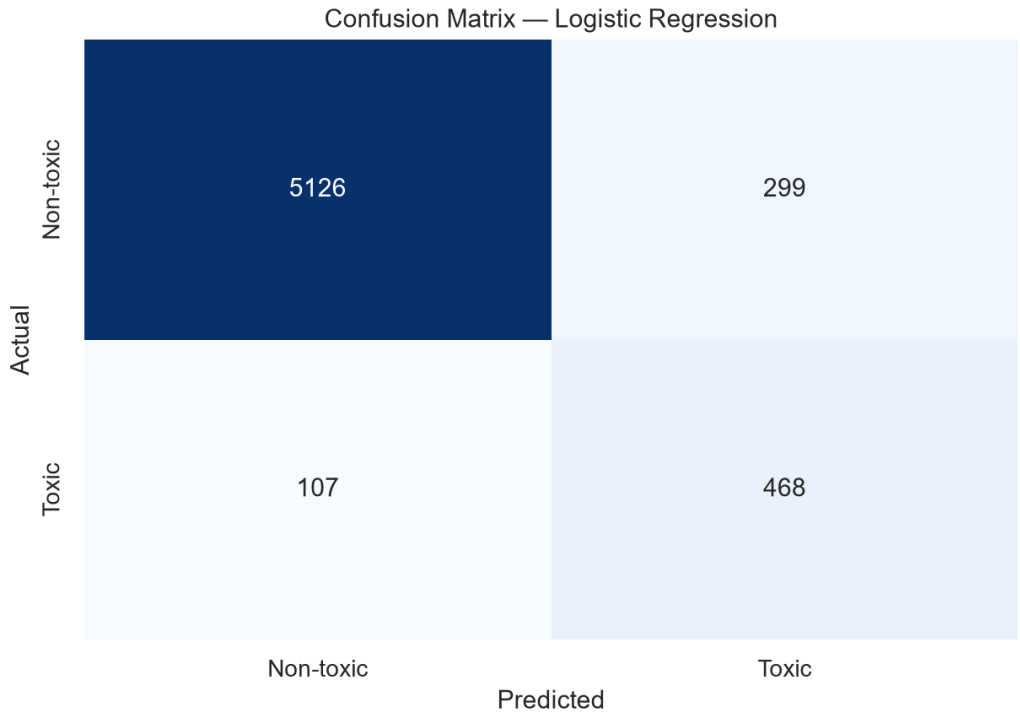


Figure 9.3 summarizes predictions for 6,000 test comments. The classifier correctly identifies 5,126 non-toxic comments and 468 toxic comments. It incorrectly flags 299 non-toxic comments and misses 107 toxic comments. These errors explain why automated flags still require human review.

9.3 Application Output

The running Streamlit application loaded Logistic Regression from the saved model files. For the test sentence “You are an idiot and nobody wants you here,” it displayed a Toxic prediction with a toxic probability of about 99.1%. This example demonstrates the interface output, not the accuracy of the model on the wider population.

Figure 9.4: Single-Comment Prediction Output

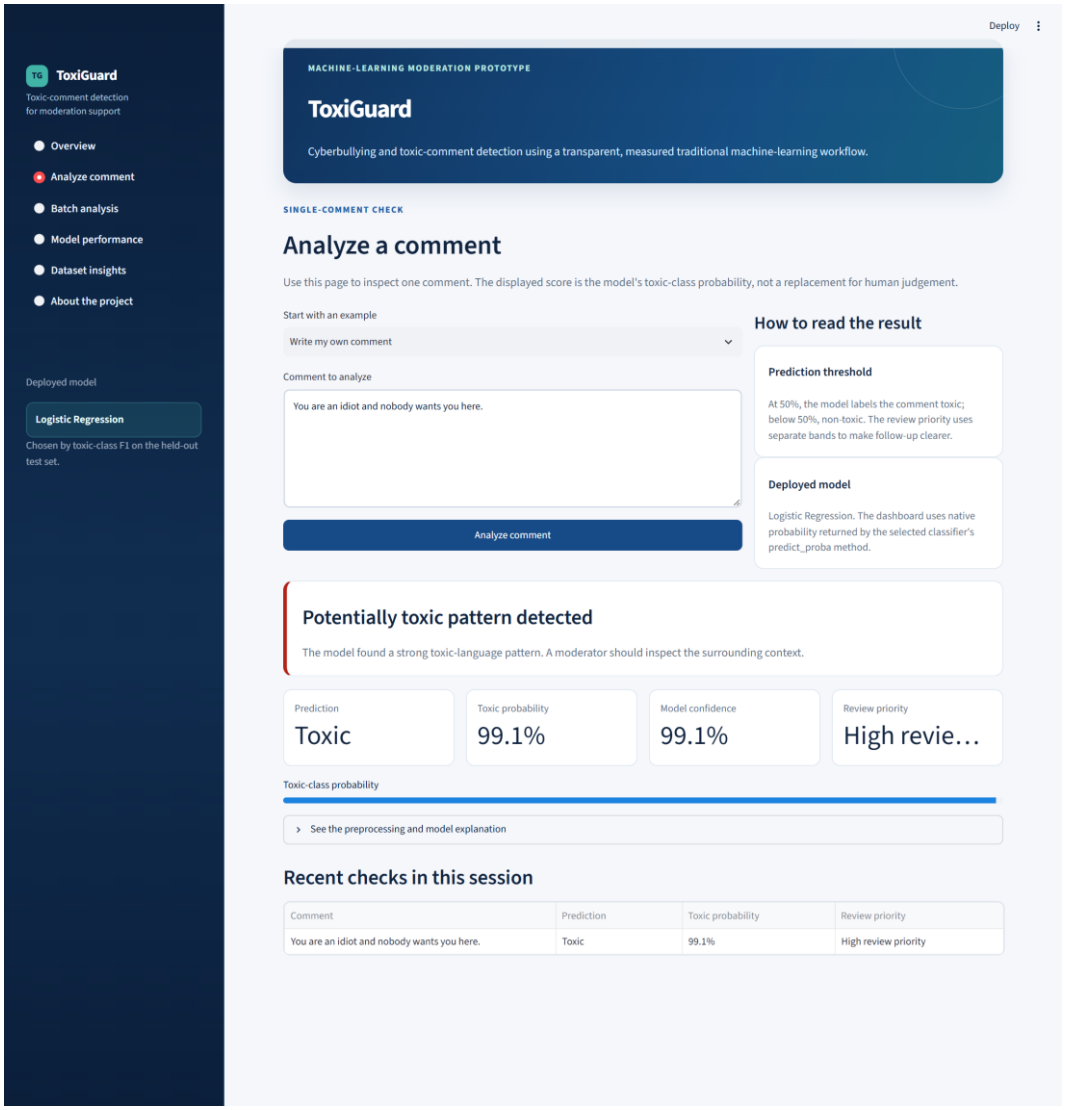


Figure 9.4 shows the entered comment, Toxic prediction, 99.1% toxic probability, and session history. The result cards communicate the model output; the review message prompts the user to consider the surrounding context before making a moderation decision.

Table 9.2: Sample Prediction Results

Sample	Toxic probability	Predicted class
Insulting sample	0.9915	Toxic
Appreciative sample	0.1427	Non-toxic
Polite disagreement	0.0501	Non-toxic

9.4 Limitations

The measured metrics arise from the held-out Jigsaw subset. The model can miss toxic comments and flag benign comments; precision 0.6102 means many flags still require review. Dataset bias, unfamiliar languages, obfuscation, quotations, and conversation context are not resolved by this binary TF-IDF model. The confidence value is a model estimate and should not be treated as a guarantee.

The same test split was used to compare and select the six models. The reported scores are therefore comparative experiment results; an independent final test set or nested validation is needed for a less biased estimate after model selection.

10. CHALLENGES AND SOLUTIONS

10.1 Class Imbalance

Only 15,293 of the 159,556 cleaned comments carry the toxic label. The split and experiment subset were stratified to retain the class ratio. The selection rule uses toxic F1 first, which reduces the chance of choosing a model solely because it predicts the majority class well.

10.2 Training and Inference Consistency

A different cleaning path in the dashboard would change the feature values presented to the trained model. ToxiGuard imports the same clean_text helper in training and inference and loads the exact vectorizer saved by training.

10.3 Interpretation and Context

Text classification cannot reliably infer intent or the full conversation context from one comment. The dashboard labels results as moderation support and describes the prediction as a prompt for human inspection. A future study should test false positives and false negatives with reviewers from the intended environment.

11. LEARNING OUTCOMES

11.1 Technical Skills Acquired

The project required a complete path from raw text to a working application. It reinforced the need to inspect class balance, fit text features only on training data, compare models using metrics that reflect the minority class, and keep preprocessing consistent between training and inference.

11.2 Practical Experience Gained

The implementation brought together CSV data preparation, feature extraction, model training, artifact serialization, and application development. Working with the saved model and vectorizer demonstrated how training outputs become reusable components of an interactive prediction system.

11.3 Technical Communication

The report and dashboard also required presenting technical results clearly. A reviewer can see the selected model, its measured tradeoffs, the confusion matrix, and examples of the interface. The paper-presentation certificate included in Appendix B records an additional opportunity to communicate related work to an academic audience.

12. CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

ToxiGuard delivers a working binary toxic-comment classifier and Streamlit dashboard. In the reproduced run, six models were compared on the same 6,000-comment test set, and Logistic Regression ranked first by toxic-class F1. Its test results were 93.23% accuracy, 61.02% toxic precision, 81.39% toxic recall, 69.75% toxic F1, and 0.9576 ROC-AUC.

The project demonstrates data preparation, TF-IDF feature extraction, measured model selection, artifact serialization, and application deployment. Its outputs can help prioritize review, while human judgment remains necessary because the model makes both false positive and false negative errors.

12.2 Limitations

Evaluation is limited to a sampled English-language dataset and a shared hold-out set used for model comparison. The system does not model conversation history or repeated behaviour. It has not been independently validated on other platforms, languages, or demographic subgroups.

12.3 Future Enhancements

- Collect feedback from human reviewers and tune the classification threshold for the intended moderation workload.
- Evaluate on comments from other platforms and languages before broader use.
- Add context-aware models and compare them with the current transparent linear baseline.
- Extend the interface to Jigsaw's separate toxicity categories after multi-label evaluation.
- Monitor model drift and document performance across different groups and language patterns.

REFERENCES

[1] Jigsaw and Conversation AI, Toxic Comment Classification Challenge, https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge (accessed 29 September 2026).

[2] scikit-learn Developers, Text feature extraction and classification documentation, https://scikit-learn.org/stable/ (accessed 29 September 2026).

[3] Streamlit, Streamlit documentation, https://docs.streamlit.io/ (accessed 29 September 2026).

[4] Python Software Foundation, Python documentation, https://docs.python.org/3/ (accessed 29 September 2026).

[5] ToxiGuard project repository, train_model.py, app.py, source data, and generated outputs, local project files examined 29 September 2026.

[6] E. Wulczyn, N. Thain, and L. Dixon, Ex Machina: Personal Attacks Seen at Scale, WWW, 2017. https://arxiv.org/abs/1610.08914

[7] T. Davidson, D. Warmsley, M. Macy, and I. Weber, Automated Hate Speech Detection and the Problem of Offensive Language, ICWSM, 2017. https://arxiv.org/abs/1703.04009

[8] L. Dixon, J. Li, J. Sorensen, N. Thain, and L. Vasserman, Measuring and Mitigating Unintended Bias in Text Classification, AIES, 2018. https://research.google/pubs/measuring-and-mitigating-unintended-bias-in-text-classification/

[9] A. Singh, ToxiGuard A Comparative Machine Learning Framework for Toxic Comment Detection and Moderation Support, research manuscript, reproduced in Appendix C.

Appendix A — Source Code

The following images show selected parts of the actual source files. The complete editable code remains in the ToxiGuard project directory.

Code Listing A.1: Shared Text-Cleaning Function

src/text_utils.py Text cleaning

```
24  def clean_text(value: object) -> str:
25      """Return the lightweight text representation used by the TF-IDF model.
26
27      The function deliberately does not remove common words or negation. In
28      short comments, words such as "not" can change meaning and are useful
29      signal for a traditional text classifier.
30      """
31      if not isinstance(value, str):
32          return ""
33
34      text = value.lower()
35      text = re.sub(r"https?://\S+|www\.\S+", " ", text)
36      text = re.sub(r"\[\[.*?\]\]", " ", text)
37      text = re.sub(r"[^a-z0-9'!?\s]", " ", text)
38      return re.sub(r"\s+", " ", text).strip()
39
```

The clean_text function lowercases comments, removes URLs and selected markup, replaces unsupported characters, and normalizes whitespace. The same helper is imported during training and application inference.

Code Listing A.2: Feature Extraction and Model Candidates

train_model.py TF-IDF and candidates

```
    sampled = stratified_sample(dataset)
    X_train, X_test, y_train, y_test = train_test_split(
        sampled["clean_text"], sampled["toxic"], test_size=0.20,
        stratify=sampled["toxic"], random_state=RANDOM_STATE,
    )
    vectorizer = TfidfVectorizer(max_features=15000, ngram_range=(1, 2),
        min_df=2, sublinear_tf=True)
    X_train_tfidf = vectorizer.fit_transform(X_train)
    X_test_tfidf = vectorizer.transform(X_test)

    candidates = {
        "Multinomial Naive Bayes": MultinomialNB(alpha=0.5),
        "Complement Naive Bayes": ComplementNB(alpha=0.5),
        "Logistic Regression": LogisticRegression(max_iter=1000,
        class_weight="balanced", random_state=RANDOM_STATE),
        "Linear SVM (Calibrated)": CalibratedClassifierCV(
            LinearSVC(class_weight="balanced", random_state=RANDOM_STATE),
            method="sigmoid", cv=3,
        ),
        "SGD Classifier": SGDClassifier(loss="modified_huber",
        class_weight="balanced", max_iter=1500, random_state=RANDOM_STATE),
        "Random Forest": RandomForestClassifier(
            n_estimators=150, max_features="sqrt", min_samples_leaf=2,
            class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE,
        ),
    }
```

The training script uses a stratified split and fits the TF-IDF vectorizer only on the training subset. The candidate dictionary defines the six classifiers compared in the experiment. Long source lines wrap visually in this listing.

Code Listing A.3: Model Selection and Serialization

train_model.py Selection and export

```
    results = pd.DataFrame(rows).sort_values(
        ["F1 Score", "Recall", "Accuracy"], ascending=False
    ).reset_index(drop=True)
    results.insert(0, "Rank", np.arange(1, len(results) + 1))
    results.to_csv(OUTPUTS / "model_comparison.csv", index=False)
    results.to_csv(OUTPUTS / "model_comparison_results.csv", index=False)
    best_name = results.loc[0, "Model"]
    best_model = fitted_models[best_name]
    best_pred = predictions[best_name]
    best_probabilities = probabilities_by_model[best_name]
    joblib.dump(best_model, MODELS / "toxiguard_model.pkl")
    joblib.dump(vectorizer, MODELS / "tfidf_vectorizer.pkl")

```

The results are ranked by toxic F1, recall, and accuracy. The script exports the comparison metrics and saves the selected classifier and TF-IDF vectorizer with joblib.

Code Listing A.4: Loading Application Artifacts

app.py Loading saved artifacts

```
@st.cache_resource(show_spinner=False)
def load_project_assets() -> tuple[object, object, dict, pd.DataFrame,
        pd.DataFrame, pd.DataFrame]:
    """Load only artifacts produced by the canonical training script."""
    required = {
        "model": MODELS / "toxiguard_model.pkl",
        "vectorizer": MODELS / "tfidf_vectorizer.pkl",
        "metadata": OUTPUTS / "project_metadata.json",
        "comparison": OUTPUTS / "model_comparison_results.csv",
        "report": OUTPUTS / "classification_report.csv",
        "top_words": OUTPUTS / "top_toxic_words.csv",
    }
    missing = [name for name, path in required.items() if not path.exists()]
    if missing:
        raise FileNotFoundError(", ".join(missing))

    model = joblib.load(required["model"])
    vectorizer = joblib.load(required["vectorizer"])
    metadata = json.loads(required["metadata"].read_text(encoding="utf-8"))
    comparison = pd.read_csv(required["comparison"]).sort_values("Rank").res
        et_index(drop=True)
    report = pd.read_csv(required["report"], index_col=0)
    top_words = pd.read_csv(required["top_words"])
    return model, vectorizer, metadata, comparison, report, top_words

```

The cached loading function checks for the model, vectorizer, metadata, and evaluation files. It loads these artifacts for reuse by the Streamlit interface, avoiding retraining during a user session.

Appendix B — Paper Presentation Certificate

The presentation certificate is reproduced with the project-title spelling corrected. It records participation at ICFC-AIDI 2026 on 5 September 2026.



Appendix C — Research Paper

ToxiGuard A Comparative Machine Learning Framework for Toxic Comment Detection and Moderation Support

Anumathi Singh

MCA 2nd year student, Department of Computer Applications, Chaitanya Bharathi Institute of Technology, Gandipet, Hyderabad, Telangana.

Email: anumathisingh@gmail.com

ABSTRACT

ToxiGuard is a text-classification prototype that identifies potentially toxic online comments and presents the results for human review. The system combines light text cleaning, term frequency–inverse document frequency features and a comparison of six traditional machine-learning classifiers. The cleaned Jigsaw dataset contains 159,556 comments, of which 15,293 carry the toxic label. A stratified subset of 30,000 comments is divided into 24,000 training instances and a 6,000-comment holdout used for model comparison and selection. Logistic Regression achieves the highest toxic-class F1 score of 0.6975, with precision of 0.6102, recall of 0.8139, accuracy of 0.9323 and ROC-AUC of 0.9576. Its confusion matrix contains 468 correctly identified toxic comments, 107 missed toxic comments and 299 false alerts. These results illustrate why accuracy alone is insufficient for this imbalanced task. A Streamlit interface supports individual comments, batch processing, CSV export and inspection of model performance. The contribution is an integrated, inspectable moderation-support workflow rather than a new classification algorithm. The experiment does not establish performance on an independent final test set or demonstrate fairness across social groups. ToxiGuard therefore treats predictions as review signals, not evidence sufficient for automatic punitive action or proof of repeated cyberbullying.

Keywords: Toxic comment detection; moderation support; TF-IDF; Logistic Regression; class imbalance; model comparison; natural language processing.

1 INTRODUCTION

1.1 Background and problem context

Online discussion allows people to exchange ideas quickly, but the same channels can carry insults, harassment and hostile responses. A moderator must distinguish disagreement from abuse while considering the surrounding conversation. As comment volume grows, reviewing every message with equal priority becomes difficult. A text classifier can help identify comments that deserve attention, provided that its limitations remain visible to the person making the decision.

ToxiGuard addresses this screening task using supervised learning. It receives a comment, transforms its text into numerical features and estimates whether the comment resembles examples labelled toxic in the training data. The application then displays a predicted class, a probability score and a review-priority band. It does not remove content, suspend accounts or determine the intent of the writer.

1.2 Scope of toxic comment detection

Toxicity, hate speech and cyberbullying overlap, but they are not interchangeable labels. A single insulting sentence can be classified as toxic without establishing a pattern of bullying. Repeated targeting, power imbalance and conversational history are outside the inputs of the present system. Although the project is motivated by cyberbullying prevention, its implemented prediction task is binary comment-level toxicity detection.

This distinction affects the interpretation of results. Correctly reproducing a dataset label is not equivalent to resolving every moderation case. Quoted abuse, reclaimed expressions, humour and criticism can require context that a short text representation does not provide. A useful prototype must therefore communicate uncertainty as well as classification output.

1.3 Motivation and objectives

The practical objective is to connect a reproducible machine-learning experiment to a working interface. The study asks which of six candidate classifiers gives the strongest toxic-class F1 score under the same feature representation and data partition. It also examines the errors behind the selected score and explains how the model is used during inference.

The project has three concrete contributions: a shared TF-IDF comparison of six classifiers; an explicit selection rule that prioritises the minority toxic class; and a dashboard that connects predictions with preprocessing, model evidence and evaluation results. These are engineering and evaluation contributions. The work does not propose a new learning algorithm or claim superiority over contextual language models that were not evaluated.

2 LITERATURE SURVEY

Waseem and Hovy [1] studied predictive features for racist and sexist language on Twitter. Their work demonstrates the importance of feature design and annotated data in abusive-language research. ToxiGuard also uses supervised text features, but the source domain and target definition differ; the present results cannot be treated as a direct comparison with their experiment.

Davidson et al. [2] examined the distinction between hate speech and offensive language. That distinction is relevant to a moderation interface: the presence of an offensive expression does not by itself identify the type, target or context of harm. ToxiGuard consequently reports a binary toxicity signal rather than presenting its output as a complete taxonomy of online abuse.

Rennie et al. [3] investigated weaknesses of conventional Naive Bayes text classification and developed a complement-based approach. Their work motivates including both Multinomial and Complement Naive Bayes as baselines. The two models provide a useful contrast in the present experiment, where one favours precision and the other favours recall.

Borkan et al. [4] proposed measures for detecting unintended bias in text classifiers. Their work highlights a limitation of relying only on aggregate evaluation: a good overall score can coexist with unequal errors for particular groups. This paper therefore distinguishes treatment of class imbalance from evaluation of social bias. No subgroup-fairness result is claimed for ToxiGuard.

The Jigsaw Toxic Comment Classification Challenge [5] supplies the project's labelled comment corpus. Scikit-learn [6] supplies the estimators and evaluation components. The methodological focus here is not the novelty of those components but their consistent use, the interpretation of their different error profiles and their integration into an inspectable application.

3 MATERIALS AND METHODS

3.1 Dataset and target definition

The local source CSV contains 159,571 rows. Removing unusable records and applying the cleaning pipeline leaves 159,556 comments: 144,263 with toxic = 0 and 15,293 with toxic = 1. Toxic comments account for approximately 9.58% of the cleaned corpus. The dataset also contains severe_toxic, obscene, threat, insult and identity_hate labels, which are available for exploratory inspection.

The canonical training script uses the existing toxic column as its binary target. It does not combine all six annotation columns into an "any harmful label" target, and it does not train six separate label predictors. Accordingly, "non-toxic" in the reported experiment means toxic = 0 under this target definition; it should not be interpreted as a guarantee that every other annotation is negative.

3.2 Data cleaning

Records missing either comment_text or the target are removed. Exact duplicates are dropped using the original comment text. The remaining text is lowercased, URLs and double-bracket Wikipedia markup are removed, unsupported characters are replaced with spaces, and repeated whitespace is normalised. Only comments whose cleaned text contains more than one character are retained for training.

The cleaner preserves English letters, digits, apostrophes, exclamation marks, question marks and whitespace. It does not apply stemming, lemmatisation or stop-word removal. Retaining words such as "not" avoids deliberately removing negation before feature extraction. However, preserving negation does not establish that the model understands its scope. The default vectorizer tokeniser also excludes standalone punctuation and one-character tokens, so punctuation retained by the cleaner is not necessarily represented as a feature.

Deduplication occurs before normalisation. Two originally different comments can therefore become identical after cleaning. The current procedure does not perform an additional cleaned-text or near-duplicate grouping step. This is an evaluation limitation, particularly when similar messages could appear on both sides of a random partition.

3.3 Sampling and partitioning

A stratified sample of 30,000 comments is selected from the cleaned corpus using random state 42. It is then split, again with stratification and the same random state, into 24,000 training comments and 6,000 holdout comments. The holdout contains 5,425 negative and 575 positive instances. Stratification preserves approximate class proportions; it does not make the classes equally frequent.

The vectorizer is fitted only on the outer training partition. Its learned vocabulary and inverse-document-frequency weights are then used to transform the holdout. All six candidates receive the same feature matrices. No oversampling, synthetic minority generation or undersampling is used after the initial stratified subset selection.

The holdout serves two purposes: comparing candidate models and choosing the deployed winner. It is therefore a model-selection holdout rather than an independent final test set. A separate untouched evaluation set would be needed to estimate the selected workflow's final generalisation performance without this selection reuse.

3.4 TF-IDF feature extraction

ToxiGuard represents comments with word unigrams and bigrams, a maximum vocabulary of 15,000 features, a minimum document frequency of two and sublinear term frequency. For a term t occurring f times in document d, the nonzero term-frequency weight is defined by Equation (1). Terms absent from a document receive zero weight.

$$tf(t,d) = 1 + ln\, f(t,d) \quad (1)$$

With n training documents and document frequency df(t), the smoothed inverse-document-frequency calculation and the unnormalised feature weight are given by Equations (2) and (3) [7]. Each resulting document vector is normalised to unit Euclidean length when it is nonzero.

$$idf(t) = ln[(1+n)/(1+df(t))] + 1 \quad (2)$$

$$v(t,d) = tf(t,d) \times idf(t) \quad (3)$$

$$x(d) = v(d) \,/\, ||v(d)||_2 \quad (4)$$

Unigrams represent individual words, whereas bigrams retain short local combinations. The representation remains sparse and does not encode full sentence structure. A new word outside the learned vocabulary contributes no feature, and a comment with no recognised features is represented by a zero vector rather than triggering new training.

3.5 Class weighting

Logistic Regression, Linear SVM, SGD and Random Forest use balanced class weights. For class c with n_c examples in a binary training set of size N, the weight is inversely proportional to its frequency, as shown in Equation (5). This increases the relative training cost associated with the minority class.

$$w_c = N \,/\, (2n_c) \quad (5)$$

Multinomial Naive Bayes uses empirical class priors, while Complement Naive Bayes estimates complement statistics. Class weighting addresses frequency imbalance, not demographic fairness. It does not remove biased annotations or ensure equal false-positive rates for identity-related language.

3.6 Logistic Regression

Logistic Regression learns a linear score $z = w^T x + b$ and maps it through the sigmoid function. It is a classifier despite its name. Positive coefficients increase the toxic-class score when the corresponding feature is present; negative coefficients decrease it.

$$p(y = 1|x) = 1 / [1 + exp(-z)] \quad (6)$$

Training minimises weighted binary log loss with L2 regularisation. For one observation, log loss is $-y \ln p - (1-y) \ln(1-p)$. The implementation sets max_iter = 1000, balanced class weights and random state 42, while retaining the estimator's default regularisation setting. Inference uses native predict_proba output. No separate calibration experiment was performed for the selected model [8].

3.7 Linear SVM with calibration

Linear SVM learns a separating hyperplane that trades margin width against classification errors. With labels encoded as −1 and +1, the configured LinearSVC uses the default squared-hinge loss. Its regularised objective has the form shown in Equation (7), with per-example weights absorbed into C_i.

$$J = \tfrac{1}{2}||w||_2^2 + \Sigma_i C_i \max(0, 1 - y_i z_i)^2 \quad (7)$$

The linear score is not itself a probability. The implementation wraps a balanced LinearSVC in CalibratedClassifierCV using sigmoid calibration and three folds. Calibration maps scores through a fitted sigmoid of the form $1/[1+\exp(Az+B)]$ [9]. The shared TF-IDF representation is fitted before this internal calibration; only the outer holdout is excluded from vocabulary fitting. This is not a fully nested text-pipeline evaluation.

3.8 SGD classifier

Stochastic gradient descent updates model parameters using successive observations rather than repeatedly evaluating the whole dataset in one batch. The project uses a linear SGDClassifier with modified Huber loss, not logistic loss. For the signed margin $m = yz$, the loss is defined piecewise as follows [10].

$$L(m) = 0, \qquad m \geq 1 \quad (8)$$
$$L(m) = (1-m)^2, \qquad -1 \leq m < 1$$
$$L(m) = -4m, \qquad m < -1$$

The update has the general form $w \leftarrow w - \eta \nabla J$, where η is the learning rate and J includes regularisation. Settings include balanced class weights, max_iter = 1500 and random state 42, with default L2 penalty and alpha = 0.0001. For this loss, binary probability estimates are derived from the clipped decision score, $p = [\text{clip}(z,-1,1)+1]/2$. These estimates should not be described as independently calibrated probabilities.

3.9 Multinomial Naive Bayes

Multinomial Naive Bayes combines a class prior with feature likelihoods under a conditional-independence assumption. ToxiGuard supplies nonnegative TF-IDF values rather than integer token counts. With feature totals N_{cj}, vocabulary size V and smoothing parameter $\alpha = 0.5$, the estimated feature probability and class score are given below [11].

$$\theta_{cj} = (N_{cj} + \alpha) / (\Sigma_k N_{ck} + \alpha V) \quad (9)$$

$$s_c(x) = \ln P(c) + \Sigma_j x_j \ln \theta_{cj} \quad (10)$$

Prediction selects the class with the largest score. Smoothing prevents an unseen class-feature combination from forcing a zero likelihood. The model is a compact baseline, but its feature-independence assumption is an approximation, especially when unigrams and overlapping bigrams are used together.

3.10 Complement Naive Bayes

Complement Naive Bayes estimates feature statistics from all classes except the candidate class. In the binary case, each class's complement is the other class. Let $N_{\neg cj}$ denote the feature total outside class c. Equation (11) gives the smoothed complement estimate; prediction favours the class with the smallest complement log-score [3, 11].

$$\theta_{\neg cj} = (N_{\neg cj} + \alpha)/(\Sigma_k N_{\neg ck} + \alpha V) \quad (11)$$

$$\hat{c} = arg\ min_c \Sigma_j x_j \ln \theta_{\neg cj} \quad (12)$$

The implementation uses $\alpha = 0.5$ and the default norm = False, so the additional complement-weight normalisation is not enabled. Complement statistics can change the minority-class decision behaviour, but they do not guarantee a better F1 score. The experiment measures this trade-off directly

rather than assuming the model will handle imbalance best.

3.11 Random Forest

Random Forest combines 150 decision trees fitted to bootstrap samples. At each split, the model considers a random subset of features, with max_features set to the square root of the vocabulary size. It uses balanced class weights and requires at least two samples in each leaf. The default Gini criterion measures node impurity [12].

$$G = 1 - \Sigma_c \, p_c{}^2 \quad (13)$$

$$p(y = 1|x) = (1/T) \, \Sigma_t \, p_t(y = 1|x) \quad (14)$$

The forest's probability is the average of the trees' class probabilities, not merely the fraction of hard votes. This nonlinear baseline can express feature interactions, although the current configuration may not exploit sparse text as effectively as a linear model. That is an interpretation of this run, not a general conclusion about all tree ensembles.

3.12 Evaluation measures

The toxic class is the positive class. TP counts correctly flagged toxic comments, FP counts non-toxic comments incorrectly flagged, FN counts missed toxic comments and TN counts correctly accepted negatives. The threshold-dependent metrics are defined in Equations (15)–(18), where N_{eval} is the total number of evaluation instances.

$$Accuracy = (TP + TN)/N_{eval} \quad (15)$$

$$Precision = TP/(TP + FP) \quad (16)$$

$$Recall = TP/(TP + FN) \quad (17)$$

$$F1 = 2TP/(2TP + FP + FN) \quad (18)$$

The receiver operating characteristic plots the true-positive rate against FP/(FP+TN) as a score threshold varies. ROC-AUC summarises this ranking behaviour. It is not the percentage of correct predictions at the dashboard's chosen threshold. Models are ranked by toxic F1, then recall and then accuracy if ties occur. No ties in F1 require those secondary criteria in the saved run.

4 SYSTEM ARCHITECTURE

The architecture separates offline learning from online inference. During training, the CSV passes through cleaning, sampling, splitting and feature extraction. Six estimators are fitted and evaluated. The selected model and the fitted vectorizer are saved together, while metrics, predictions and plots are stored as evaluation artefacts. Figure 1 summarises the two paths.

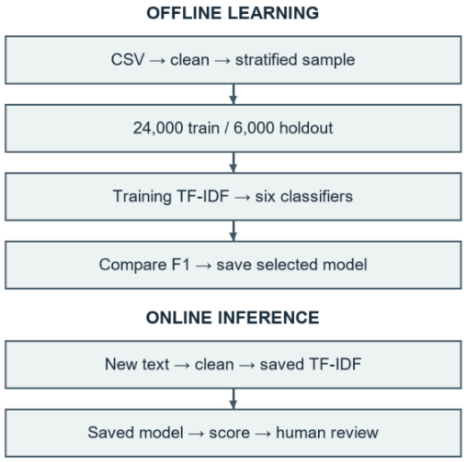


Figure 1. Offline comparison and online inference architecture.

At inference time, a new comment is cleaned using the same utility, transformed with the saved vocabulary and passed to the saved classifier. The application does not refit the vectorizer on user submissions. This separation keeps the meaning and order of feature columns consistent between training and prediction.

The interface is a presentation layer over these artefacts. Its performance page displays the saved experiment; it does not estimate new test accuracy from demonstration comments. Likewise, adding a comment to the session history does not update the trained model.

5 UML AND DATA FLOW DIAGRAMS

5.1 Use case model

The principal user is a moderator or presenter who submits comments, reviews predictions and inspects the experiment. A maintainer runs the training script and manages saved artefacts outside the dashboard. Figure 2 separates these responsibilities; it does not imply that the prototype implements authentication or role-based access control.

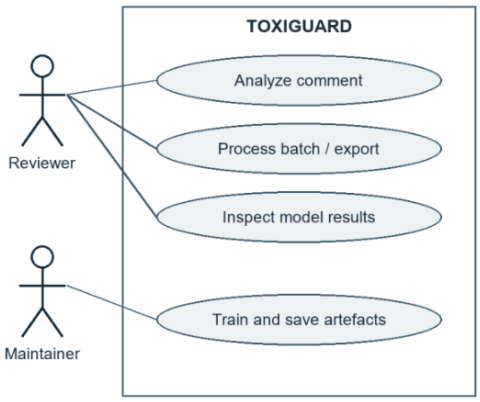


Figure 2. Use cases for the reviewer and project maintainer.

5.2 Activity model

Figure 3 shows the single-comment activity. Empty cleaned input produces a warning. Otherwise, the application computes a probability, applies the class threshold and assigns a review-priority band. A human remains responsible for interpreting the text in context and deciding whether action is warranted.

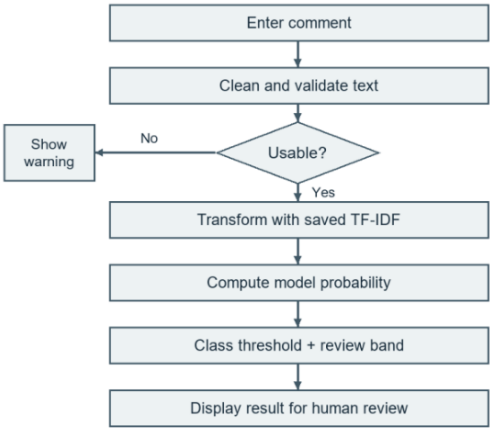


Figure 3. Single-comment analysis activity.

5.3 Data flow

The inference data flow contains an external comment source, a preprocessing step, saved model resources and a result destination. Figure 4 also identifies session history and optional CSV export. It describes data movement within the prototype, not an external social-media integration or a persistent moderation case-management system.

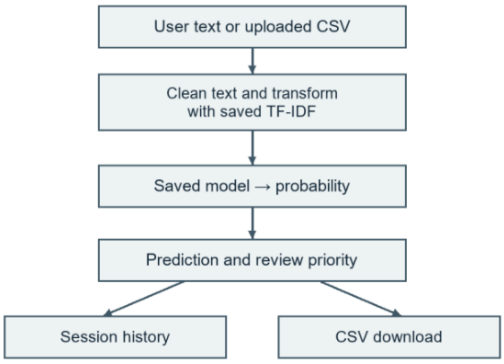


Figure 4. Inference data flow and output destinations.

6 IMPLEMENTATION

6.1 Software organisation

The project uses Python, pandas, scikit-learn, joblib, matplotlib, seaborn and Streamlit. The canonical train_model.py script produces cleaned data, comparison results, the classification report, plots and model files. The shared text utility supplies the cleaning function, while app.py loads the saved resources and serves the interface.

The dashboard provides Overview, Analyze comment, Batch analysis, Model performance, Dataset insights and About the project pages. Its overview connects dataset counts with the selected model and the selection rule. Figure 5 shows the running application. The displayed dataset total describes the cleaned corpus, not the number of instances actually used to fit the model.

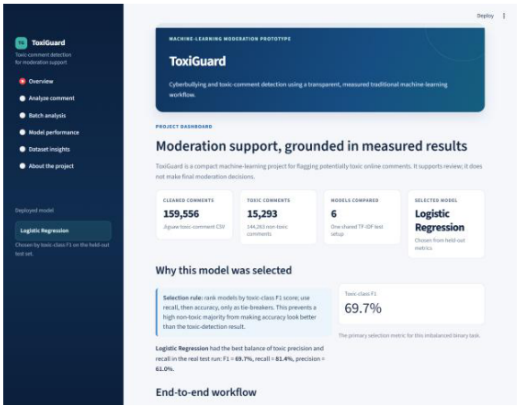


Figure 5. Current ToxiGuard dashboard and experiment summary.

6.2 Single-comment inference

The analysis page accepts free text or a built-in example. It applies the shared cleaner,

transforms the resulting text and obtains the positive-class probability. A score of at least 0.50 produces the Toxic label. The interface also exposes the cleaned text and, for a linear model with accessible coefficients, up to five present features with positive contributions $w_j \times x_j$.

These feature contributions describe the fitted linear score. They are not causal explanations, fixed word-ban rules or proof that a particular expression is always abusive. Negative evidence and the intercept also influence the probability. A short list of positive terms therefore cannot explain every aspect of the decision.

6.3 Batch processing and session history

The batch page accepts either a CSV file or one comment per line. For uploaded files, the user selects the text column; comment_text is preferred when present. Missing values are removed, and comments that become empty during cleaning are skipped. Each operation processes at most the first 1,000 submitted comments.

The output contains the comment, predicted class, toxic probability, displayed confidence and review priority. The user can download these records as CSV, a plain-text tabular format with comma-separated fields. Session history retains a limited set of recent checks for display. It is not a durable audit log, an annotation database or a feedback-training mechanism.

6.4 Review priority and probability interpretation

Review priority is separate from the binary class threshold. Scores below 0.45 receive Lower review priority; scores from 0.45 to below 0.75 receive Review recommended; scores of at least 0.75 receive High review priority. Thus, a comment between 0.45 and 0.50 can be labelled non-toxic and still be recommended for review.

These bands are interface heuristics, not validated estimates of harm severity. The displayed confidence is $\max(p,1-p)$, where p is the model's toxic probability. It is not the measured probability that a moderation decision is correct. Native probability output, particularly from a class-weighted model, does not establish calibration on the deployment population.

7 RESULTS AND DISCUSSION

7.1 Dataset distribution

Figure 6 shows the cleaned corpus distribution. The negative class is about 9.43 times as frequent as the positive class. The 6,000-comment holdout has a similar imbalance. A classifier predicting only the negative class would obtain 90.42% accuracy on that holdout while detecting no toxic comments. This baseline explains the decision to prioritise toxic F1.

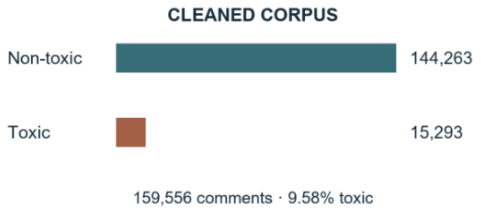


Figure 6. Cleaned dataset counts from the project metadata.

7.2 Six-model comparison

Table 1 reports toxic-class precision, recall and F1 on the common holdout. LR denotes Logistic Regression, SGD the modified-Huber classifier, SVM the calibrated Linear SVM, MNB Multinomial Naive Bayes, CNB Complement Naive Bayes and RF Random Forest. Scores are rounded to four decimal places. Figure 7 presents the F1 ranking without combining different metrics into a stacked total.

Table 1. Toxic-class precision, recall and F1.

Model	Precision	Recall	F1
LR	**0.6102**	**0.8139**	**0.6975**
SGD	0.6134	0.7948	0.6924
SVM	0.8549	0.5739	0.6868
MNB	0.9263	0.4591	0.6140
CNB	0.4667	0.8643	0.6061
RF	0.5580	0.6278	0.5908



Figure 7. Toxic-class F1 scores from the saved comparison run.

Logistic Regression has the highest F1, but different models lead on different criteria. Multinomial Naive Bayes has the highest precision, 0.9263, while detecting fewer than half of the toxic comments. Complement Naive Bayes has the highest recall, 0.8643, but its precision is only 0.4667. These results demonstrate different error trade-offs rather than a single ordering that holds for every moderation objective.

Table 2 gives accuracy and ROC-AUC for the same predictions and probability scores. Linear SVM has the highest accuracy, 0.9498, whereas Logistic Regression has the highest ROC-AUC, 0.9576. A model's high accuracy therefore does not establish the strongest toxic-class detection at its operating threshold.

Table 2. Overall accuracy and ROC-AUC.

Model	Accuracy	ROC-AUC
LR	**0.9323**	**0.9576**
SGD	0.9323	0.9533
SVM	0.9498	0.9555
MNB	0.9447	0.9449
CNB	0.8923	0.9449
RF	0.9167	0.9141

7.3 Selected model and error counts

The selected Logistic Regression model correctly classifies 5,594 of 6,000 comments. Its confusion matrix is shown in Figure 8. Of the 575 toxic comments, 468 are detected and 107 are missed. Of the 5,425 negative comments, 299 are incorrectly flagged and 5,126 are correctly classified.

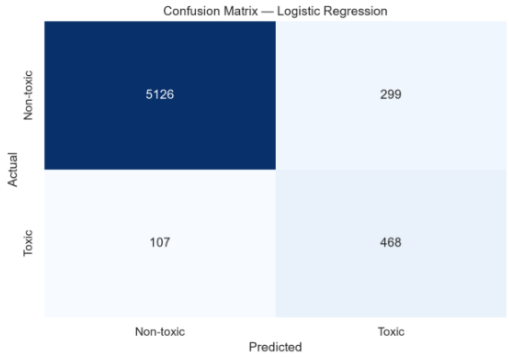


Figure 8. Project-generated confusion matrix for Logistic Regression.

The model produces 767 positive predictions. Approximately 39.0% of those alerts are false positives under the dataset labels, consistent with precision of 61.02%. Its false-negative rate is 18.61%, and its false-positive rate among negative comments is 5.51%. These quantities are more informative for a reviewer than accuracy alone because they describe both missed content and unnecessary review.

The F1 difference between Logistic Regression and SGD is approximately 0.0050, or half a percentage point. The saved experiment contains one random partition and does not provide confidence intervals or a paired significance test. Logistic Regression is consequently the measured winner of this run, not a statistically established universal winner. The same holdout was used for selection, which further limits generalisation claims.

7.4 Application demonstration

Figure 9 shows the actual result for the built-in-style example "You are useless and nobody wants to hear you." The deployed model returns a toxic probability of approximately 72.1%, a Toxic label and Review recommended. The label and priority differ in wording because they use different thresholds, not because the application is inconsistent.

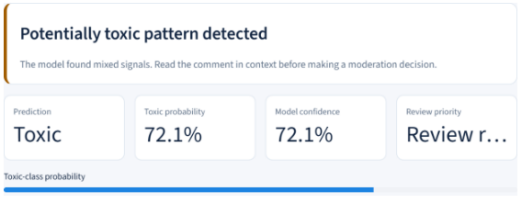


Figure 9. Live single-comment result from the saved Logistic Regression model.

Figure 10 shows batch processing of three demonstration comments. The constructive example receives a probability of approximately 10.9%; the hostile example receives 72.1%; and the context-sensitive criticism receives 17.3%. These examples demonstrate the interface and export path only. They are not an additional labelled benchmark and do not establish accuracy, fairness or robustness.

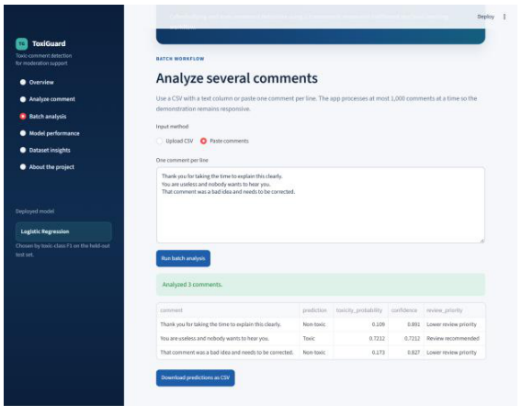


Figure 10. Batch workflow with three illustrative comments and CSV export.

7.5 Bias and responsible interpretation

The system addresses class imbalance through stratified partitioning, balanced weights in four candidates and minority-class metrics. These measures must not be described as eliminating bias against particular communities. The training labels may reflect annotation assumptions, and lexical features may associate identity terms or dialect patterns with toxicity even when the surrounding statement is benign.

No identity-subgroup annotations, counterfactual identity substitutions or subgroup error-rate analysis were used in this experiment. The current evidence therefore supports a statement about aggregate toxic-label prediction, not social fairness. A future audit should examine false positives on neutral identity mentions, quotation, counterspeech and dialect variation, alongside subgroup-oriented metrics such as those discussed by Borkan et al. [4].

7.6 Limitations and validity

The experiment is restricted to a 30,000-comment subset, one random seed and one source domain. The selected model is fitted on 24,000 instances rather than the entire cleaned corpus. No independent external corpus, temporal split, user-level grouping or final untouched test set is available in the reported evaluation. Near-duplicate contamination after normalisation has not been ruled out.

The ASCII-oriented cleaner limits multilingual coverage. Word unigrams and bigrams offer only local context and can miss sarcasm, obfuscated spellings, long-range negation and conversational intent. The system does not verify whether a sequence of messages constitutes sustained bullying, nor does it predict separate threat, insult or identity-hate labels.

Probability reliability and review thresholds have not been validated against deployment prevalence or reviewer costs. ROC-AUC does not resolve those questions. The project also lacks measured latency, memory, concurrency and moderation-workload benchmarks, so no production-scale or real-time performance claim is made. Human review remains necessary, but its effect on outcomes was not evaluated in a user study.

8 CONCLUSION

ToxiGuard connects a traditional text-classification experiment to a working moderation-support interface. Using a shared 15,000-feature TF-IDF representation, six classifiers were compared on a stratified 6,000-comment holdout. Logistic Regression achieved the highest toxic-class F1 of 0.6975, with recall of 0.8139 and precision of 0.6102. Its 299 false alerts and 107 missed toxic comments show why the result should be interpreted as a screening capability rather than a final moderation judgement.

The main outcome is an inspectable workflow with explicit preprocessing, model selection,

stored evaluation artefacts and single-comment and batch inference. The experiment supports this prototype's measured comparison, while leaving independent generalisation, probability calibration, contextual understanding and social fairness as unresolved evaluation needs.

9 FUTURE ENHANCEMENT

The next evaluation should reserve an untouched final test set and use cross-validation within the remaining development data. Text normalisation and duplicate grouping should occur within a leakage-aware protocol, and the vectorizer should be included in the cross-validation pipeline. Repeated runs and uncertainty estimates would help determine whether the small F1 differences between leading models are stable.

Subsequent work can compare character features and contextual language models, extend the target to multiple toxicity labels, and evaluate multilingual data with an appropriate cleaner. Calibration curves, precision–recall analysis and validation-based threshold selection should precede operational use of probability bands. A subgroup-fairness audit and a reviewer study are also needed before claiming equitable or effective moderation support. Any feedback collection should include consent, retention controls and a documented process for correcting labels.

REFERENCES

[1] Z. Waseem and D. Hovy, "Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter," NAACL Student Research Workshop, pp. 88–93, 2016. doi: 10.18653/v1/N16-2013.

[2] T. Davidson, D. Warmsley, M. Macy and I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," ICWSM, vol. 11, no. 1, pp. 512–515, 2017. doi: 10.1609/icwsm.v11i1.14955.

[3] J. D. M. Rennie, L. Shih, J. Teevan and D. R. Karger, "Tackling the Poor Assumptions of Naive Bayes Text Classifiers," ICML, pp. 616–623, 2003. https://people.csail.mit.edu/jrennie/papers/icml03-nb.pdf

[4] D. Borkan, L. Dixon, J. Sorensen, N. Thain and L. Vasserman, "Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification," 2019. arXiv:1903.04561. https://arxiv.org/abs/1903.04561

[5] Jigsaw and Conversation AI, "Toxic Comment Classification Challenge," Kaggle, 2018. https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[7] Scikit-learn developers, "TfidfTransformer," API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html

[8] Scikit-learn developers, "Linear Models," User Guide. https://scikit-learn.org/stable/modules/linear_model.html

[9] Scikit-learn developers, "Probability calibration," User Guide. https://scikit-learn.org/stable/modules/calibration.html

[10] Scikit-learn developers, "SGDClassifier," API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

[11] Scikit-learn developers, "Naive Bayes," User Guide. https://scikit-learn.org/stable/modules/naive_bayes.html

[12] Scikit-learn developers, "RandomForestClassifier," API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Appendix D — Project Outputs

The files below are the direct outputs of the reproduced training run. They can be inspected in the ToxiGuard project directory.

Table D.1: Project Output Files

File	Purpose
project_metadata.json	Run settings and selected-model summary
model_comparison.csv	Six-model held-out comparison
classification_report.csv	Per-class precision, recall, F1, and support
predictions_output.csv	Row-level test predictions and probabilities
eda_plots.png	Dataset exploration figures
model_comparison.png	Model performance chart
confusion_matrix.png	Errors for the selected classifier

The output files are in D:\ToxiGuard\outputs. The saved classifier and TF-IDF vectorizer are in D:\ToxiGuard\models.

Recorded training command: python train_model.py

Dashboard command: python -m streamlit run app.py